

CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Jenkins I



WE WANT
SKILLS!

mundosE

Manual para el Alumno: Jenkins I.....	2
Objetivo del encuentro:.....	2
1. Introducción a Jenkins.....	3
2. Arquitectura y funcionalidades.....	4
3. Instalación y configuración básica.....	5
4. Creación y gestión de <i>jobs</i>.....	6
4.1 Estructura del formulario.....	6
4.2 <i>Builds</i> parametrizados.....	7
4.3 Encadenamiento y orquestación.....	7
4.4 Gestión de artefactos y retención.....	7
4.5 Visibilidad y <i>dashboards</i>	8
4.6 <i>Freestyle</i> vs. <i>Pipeline</i>	8
4.7 Buenas prácticas resumidas.....	8
Bibliografía.....	10

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este presentamos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Jenkins I

Objetivo del encuentro:

1. Explicar el origen histórico de Jenkins y su relevancia en los flujos DevOps contemporáneos.
2. Describir la arquitectura maestro-agente y los principales componentes internos (*controller, queue, executor, workspace*).
3. Enumerar los pasos esenciales para instalar Jenkins LTS sobre una distribución GNU/Linux y realizar la configuración inicial segura.
4. Crear y ejecutar un **Job Freestyle** que compile, pruebe y archive artefactos, aplicando buenas prácticas básicas

1. Introducción a Jenkins

Desde su concepción como *side-project* dentro de Sun Microsystems en 2005, Jenkins —inicialmente Hudson— se diseñó con una premisa simple: aliviar la «fatiga de integración» que experimentaban los equipos cuando acumulaban cambios de código durante días. La filosofía de *integración continua* que hoy consideramos estándar estaba lejos de ser una práctica común; los desarrolladores solían fusionar ramas grandes al final de un ciclo, provocando conflictos difíciles de resolver y largas jornadas de “caza de bugs”. A lo largo de 2010, la compra de Sun por parte de Oracle generó incertidumbre sobre el futuro de Hudson. La comunidad decidió conservar el ADN abierto del proyecto y lo bifurcó bajo el nombre **Jenkins**; el 29 de enero de 2011 se anunció oficialmente el *fork*, tras una votación que reunió a cientos de contribuidores en menos de 72 h. Este hito evidenció el valor de la colaboración abierta: en los siguientes dos años Jenkins multiplicó por diez su catálogo de *plugins* y se convirtió en la herramienta de CI/CD más popular del mundo, con adopción temprana en proyectos como Android, Ruby on Rails y OpenStack.

En esencia, Jenkins automatiza un flujo que puede describirse con el acrónimo **C-B-T-P-D** (*Checkout, Build, Test, Package, Deploy*). Al detectar un *commit* —por *polling* o *webhook*—, clona el repositorio en un *workspace* aislado, ejecuta los comandos de compilación, corre pruebas unitarias y, si todo pasa, empaqueta el artefacto y lo deja listo para ser publicado o desplegado. Cada paso es visible en la interfaz web, generando transparencia y trazabilidad: se sabe qué versión del código produjo cada binario, qué parámetros se usaron y quién aprobó la promoción. Esta trazabilidad reduce significativamente el **Mean Time To Detect (MTTD)** y el **Mean Time To Repair (MTTR)** cuando aparece un problema en producción.

La popularidad de Jenkins radica en cuatro pilares: **extensibilidad, neutralidad tecnológica, autogestión** y **comunidad**. Con más de 1.800 *plugins*, cualquier lenguaje o servicio se integra: desde pruebas con JUnit en Java hasta despliegues de contenedores en Kubernetes, análisis estático con SonarQube o notificaciones Slack. Su modelo *self-hosted* brinda control total sobre la infraestructura, a diferencia de soluciones SaaS, que pueden imponer límites de ejecución o almacenar datos en regiones no deseadas. Finalmente, la comunidad global —con eventos como Jenkins World y *hacktoberfests* mensuales— garantiza soporte, documentación y una evolución constante.

Comparativa rápida. Mientras GitHub Actions o GitLab CI ofrecen *pipelines* en la nube “listos para usar”, Jenkins sobresale cuando se requiere personalización profunda (scripts Groovy, condiciones de seguridad estrictas, nodos en redes privadas) o cuando los costos de minutos en SaaS se disparan en organizaciones con miles de *builds* diarios.

Indicadores clave según DORA muestran que los equipos con CI/CD madura despliegan 46 × más rápido y tienen 5 × menos fallas *pos-release*. Jenkins, junto con un *pipeline* bien diseñado, es la base para alcanzar esos números.

Recursos para profundizar:

- A. **Jenkins** © (2025). Construye grandes cosas a cualquier escala.
<https://www.jenkins.io/>
- B. **Accelerate State of DevOps Report** (Informe DORA) (2024) (Cap. 3: métricas de desempeño): <https://cloud.google.com/devops/state-of-devops/>
- C. **Liuzzo, J.** (2025, ene 10). Revolutionizing DevOps with Jenkins. *CloudBees*.
<https://www.cloudbees.com/blog/devops-and-jenkins>

2. Arquitectura y funcionalidades

La arquitectura de Jenkins puede imaginarse como un hormiguero: un **controller** (maestro) que organiza la cola de trabajo y múltiples **agentes** (obreras) que ejecutan las tareas. Cuando el *controller* recibe un nuevo *build request*, lo coloca en la **Build Queue**. Cada agente expone uno o más **executors** (hilos) que, al quedar libres, toman el siguiente trabajo compatible con su **label** (etiqueta). Este diseño facilita la paralelización horizontal: agregar capacidad es tan simple como registrar un nuevo agente, ya sea una VM, un contenedor o un *pod* efímero en Kubernetes.

Componentes principales:

- **Controller**: hospeda la UI y la lógica de orquestación; persiste configuraciones en XML dentro de `$JENKINS_HOME`.
- **Agent**: nodo externo que corre el agente *remoting* JAR; puede iniciarse por SSH, JNLP o Docker.
- **Workspace**: carpeta temporal donde se clona el código y se generan artefactos.
- **Build Queue**: lista FIFO (con prioridad) de trabajos pendientes.
- **Plugin Manager**: descarga, instala y actualiza *plugins* desde el Update Center.

Seguridad por capas. Jenkins separa **autenticación** y **autorización**. Para autenticación puede delegar en LDAP, GitHub OAuth, SAML o su base interna. Para autorización ofrece Matrix-based (roles por usuario/grupo), Project-based (ACL por job) y Role-Strategy Plugin (roles jerárquicos). Todas las credenciales se cifran con AES-128 y una clave maestra almacenada en el controller; el archivo `credentials.xml` nunca debería exponerse.

Escalabilidad. En instalaciones medianas (100–200 *jobs* diarios) un *controller* con 4 CPU y 8 GiB RAM es suficiente. Cuando las colas superan los 5 min, se recomienda externalizar agentes en Kubernetes usando **kubernetes-plugin**: cada *build* se ejecuta en un *pod* aislado, destruyéndose al finalizar y liberando recursos.

Observabilidad. El **Prometheus Plugin** expone métricas como `jenkins_queue_size_value`, `jenkins_job_duration_seconds` y `jenkins_executor_utilization`. Con Grafana se construyen *dashboards* que alertan si la cola supera cierto umbral o si la duración media de *builds* crece semana a semana.

Recursos para profundizar:

- A. **Jenkins** © (2025). *Managing Nodes*.
<https://www.jenkins.io/doc/book/managing/nodes/>
- B. **Jenkins** © (2025). *Securing Jenkins*.
<https://www.jenkins.io/doc/book/system-administration/security/>

3. Instalación y configuración básica

Instalar Jenkins implica satisfacer prerequisites (Java 17), agregar el repositorio oficial, iniciar el servicio y ejecutar el **Setup Wizard**. El wizard solicita la clave ubicada en `/var/lib/jenkins/secrets/initialAdminPassword` y luego permite instalar *plugins* recomendados y crear el usuario administrador. Para entornos automáticos se emplea **Jenkins Configuration as Code (JCasC)**, un *plugin* que carga un archivo YAML con toda la configuración inicial: usuarios, credenciales, jobs semilla y hasta agentes permanentes.

Prerrequisitos: 2 CPU, 4 GiB RAM (mínimo); Java 17 OpenJDK; puerto 8080 libre. En distribuciones basadas en Debian, se importa la clave GPG y se instala con `apt`. En Red Hat, se usa `yum`. Para contenedores, la imagen oficial `jenkins/jenkins:lts-jdk17` está endurecida y lista para correr con un volumen persistente.

Primera configuración: tras el wizard, se recomienda habilitar **CSRF Protection**, configurar la zona horaria (`America/Argentina/Buenos_Aires`) y ajustar la política de retención de builds (p. ej. 30 días o 50 builds). Las credenciales deben clasificarse por carpetas (plugin *Credentials View*).

Backup: copiar `$JENKINS_HOME` con rsync o snapshots EBS; para granularidad, use **ThinBackup Plugin** que programa copias diferenciales.

Actualizaciones: seguir el canal LTS y probar en un entorno *staging*. Al actualizar,

revisar los *breaking changes* de *plugins*.

Recursos para profundizar:

- A. **Jenkins** © (2025). *Linux*. <https://www.jenkins.io/doc/book/installing/linux/>
- B. **Docker** © (2025). *Repositorio Docker Hub Jenkins*.
<https://hub.docker.com/r/jenkins/jenkins>

4. Creación y gestión de *jobs*

El **Job Freestyle** constituye la unidad didáctica ideal para asimilar la mecánica interna de Jenkins, porque cada pestaña del formulario de configuración se correlaciona con una fase del ciclo CI/CD. A diferencia de los **Pipelines Declarativos**, cuyo flujo se codifica en un archivo **Jenkinsfile**, el Job Freestyle se define mediante la UI y guarda su configuración en un XML dentro de `$JENKINS_HOME/jobs/<job>/config.xml`. Comprender esta “vieja escuela” es provechoso por dos motivos:

- (1) ilustra la anatomía de un build sin distracciones de sintaxis Groovy; y,
- (2) muchos *plugins* aún exponen opciones únicamente para Freestyle.

4.1 Estructura del formulario

Sección	Propósito	Buenas prácticas
General	Nombre, descripción, política de descarte de <i>builds</i> y enlaces a repositorios.	Activar “Discard old builds” (30 días / 50 builds) y, si el proyecto está en GitHub, marcar la casilla “GitHub Project” para habilitar vistas pull-request.
Source Code Management (SCM)	URL y credenciales del repositorio, ramas a compilar.	Usar credenciales tipo <i>SSH key</i> ; setear <code>*/main</code> y, si hay <i>releases</i> , incluir <code>origin/release/*</code> .
Build Triggers	Define cuándo se dispara el <i>job</i> : <i>cron</i> , <i>webhooks</i> , <i>upstream</i> .	Preferir <i>GitHub webhook</i> o <i>Bitbucket webhook</i> a <i>Poll SCM</i> para evitar consultas vacías.

Build Environment	Variables de entorno, borrado de <i>workspace</i> , <i>wrappers</i> de credenciales.	Habilitar “ <i>Delete workspace before build starts</i> ” para <i>builds</i> reproducibles.
Build Steps	Comandos Shell, Gradle, Maven, Ant, Xcode.	Mantener scripts en repositorio y referenciar <code>./ci/build.sh</code> para versionar lógica.
Post-Build Actions	Archivar artefactos, publicar reportes, notificar.	Archivar <code>target/*.jar</code> y publicar resultados JUnit para retroalimentación instantánea.

4.2 Builds parametrizados

Marcar “**This build is parameterized**” permite inyectar valores variables en tiempo de ejecución: *String Parameter* (`ENV=staging`), *Choice Parameter* (`dev | qa | prod`) o *Boolean Parameter* (`SKIP_TESTS`). Los parámetros se exponen como variables de entorno (`$ENV`) dentro de los *Build Steps*, facilitando *pipelines* polimórficos.

Caso práctico: un mismo Job Freestyle empaqueta una API para distintos entornos. El paso de compilación lee `mvn -P$ENV clean package` y el Post-Build sube el JAR al prefijo `s3://my-bucket/$ENV/`.

4.3 Encadenamiento y orquestación

Antes de la era Pipeline, se encadenaban *jobs* con la opción “**Build other projects**” (*downstream*) o a través del *plugin* **Parameterized Trigger**. Aunque hoy se prefiere Pipeline Declarativo, conocer esta mecánica ayuda a mantener proyectos legados. Dos patrones clásicos:

1. **Build → Test → Deploy:** Job A compila, Job B testa, Job C despliega. Cada enlace pasa artefactos mediante **Copy-Artifact Plugin**.
2. **Fan-in/Fan-out:** Job padre dispara dos hijos en paralelo y espera su finalización para continuar.

4.4 Gestión de artefactos y retención

Un artefacto es cualquier archivo relevante que debe preservarse (JAR, WAR, zip, reporte). Marcar “Archive the artifacts” copia los archivos al directorio

`builds/<n>/archive` y los sirve vía HTTP. El **Fingerprinting** calcula un *hash* SHA-1 y permite rastrear qué versión se desplegó en QA o Prod.

Política sugerida: retener 30 artefactos exitosos y 10 fallidos; versiones antiguas pueden regenerarse mediante *rebuild* si se conserva el código.

4.5 Visibilidad y *dashboards*

Con el *plugin* **Build Monitor View** se crea un panel a modo de semáforo (verde/rojo) que facilita visualizar salud del proyecto. Para múltiples equipos se agrupan *jobs* en **Folders** y se asignan roles con **Role Strategy Plugin**, evitando colisiones de permisos.

4.6 Freestyle vs. Pipeline

Aspecto	Freestyle	Declarative Pipeline
Configuración	UI XML	<code>Jenkinsfile</code> versionado
Paralelismo	Limitado (multi-config)	<code>parallel {}</code> nativo
Portabilidad	Copiar config.xml	Git clone = pipeline listo
Mantenimiento	Click-ops propenso a <i>drift</i>	Código revisado en PR

La tendencia de la industria es migrar a Pipeline As Code, pero Freestyle sigue siendo útil para *proof-of-concepts* rápidos y para trabajos utilitarios (limpieza de *workspaces*, *backups*). Entenderlo ayuda a depurar errores cuando un paso falla antes de que el equipo haya convertido TODO su flujo a Declarative.

4.7 Buenas prácticas resumidas

1. **Nombrar claramente:** `<equipo>-<servicio>-< tarea>`.
2. **Evitar lógica compleja** en Shell dentro del Job; mover a *scripts* versionados.
3. **No almacenar credenciales** en texto plano; usar *Credentials Binding*.
4. **Revisar retención** para no llenar disco.
5. **Gestionar dependencias:** corre *builds* en contenedores para reproducibilidad.

Recursos para profundizar:

- A. **Jenkins** © (2025). *Pipeline*. (Guía “Freestyle Projects”).
<https://www.jenkins.io/doc/book/pipeline/>
- B. **GitHub** © (2025). Copy-Artifact Plugin (GitHub):
<https://github.com/search?q=Copy%E2%80%91Artifact%20Plugin%20&type=repositories>

Bibliografía

García, M. (2021). *Introducción a Cloud Computing: Modelos de Servicios, Implementaciones y Estrategias Empresariales*. Ed. Alfaomega. ISBN: 978-6075385687.

Google Cloud Platform Documentation. (2024). *Documentación Oficial de Google Cloud Platform*. Google LLC. <https://cloud.google.com/docs?hl=es-419>

Google Cloud Compute Engine Documentation. (2024). *Compute Engine – Máquinas virtuales en GCP*. Google LLC. <https://cloud.google.com/compute/docs?hl=es-419>

Google Cloud SQL Documentation. (2024). *Bases de datos gestionadas en Google Cloud*. Google LLC. <https://cloud.google.com/sql/docs?hl=es-419>

Google Cloud Run Documentation. (2024). *Ejecución serverless de contenedores*. Google LLC. <https://cloud.google.com/run/docs?hl=es-419>

Google Cloud Storage Documentation. (2024). *Almacenamiento de objetos en la nube*. Google LLC. <https://cloud.google.com/storage/docs?hl=es-419>

Google BigQuery Documentation. (2024). *Servicio de análisis de datos a gran escala*. Google LLC. <https://cloud.google.com/bigquery/docs?hl=es-419>

Rosenberg, J. & Mateos, J. (2022). *Practical Google Cloud Platform: Harnessing Google's Infrastructure for Business Success*. Ed. Apress. ISBN: 978-1484274960.

Joshi, S. (2020). *Cloud Computing: Concepts, Technology & Architecture*. Ed. Pearson Education. ISBN: 978-9353947051.

Stackscale (2023). *Modelos de servicio Cloud Computing*. Stackscale Blog. <https://www.stackscale.com/es/blog/modelos-de-servicio-cloud>